

SNARKeling Treasure Hunt Audit Report

Fady Efat (Darkin)

April 28, 2026



SNARKeling Treasure Hunt Audit Report

Version 1.0

Fady Efat (Darkin)

April 28, 2026

SNARKeling Treasure Hunt Audit Report

Fady Efat (Darkin)

April 28, 2026

Prepared by: Fady Efat (Darkin) Lead Auditors: - Fady Efat (Darkin)

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
- High
- Medium
- Low

Protocol Summary

A smart contract application for a zero-knowledge treasure hunt. Users can submit proofs to claim treasures, and the protocol tracks claimed hashes to prevent double-spending.

Disclaimer

The Fady Efat (Darkin) team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity and Noir implementation of the contracts.

Risk Classification

		Impact		
Likelihood		High	Medium	Low
	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: N/A
- Solc Version: 0.8.20 (or equivalent)
- Chain(s) to deploy contract to: Ethereum

Scope

```
./src/  
#-- TreasureHunt.sol  
#-- main.nr
```

Roles

- Owner: Administers the protocol, can withdraw unclaimed funds, update verifier, and pause/unpause.
- Participant: Submits zero-knowledge proofs to claim treasures.

Executive Summary

Issues found

Severity	Number of issues found
High	2
Medium	1
Low	1
Informational	0
Total	4

Findings

High

[H-1] Critical Logic Error in Variable Reference Allows Infinite Replay Attacks

Description

Root Cause: The primary vulnerability stems from a critical logic error in the `claim()` function of `TreasureHunt.sol`. The double-spend prevention check incorrectly references an uninitialized immutable variable `_treasureHash` (set to 0), instead of the function parameter `treasureHash`. This allows attackers to repeatedly claim the same treasure hash without restriction.

```
// Root cause in the codebase with @> marks to highlight the relevant section
function claim(bytes calldata proof, bytes32 treasureHash, address payable recipient) external {
    // @> Missing verification: require(!claimed[treasureHash], "Already Claimed");

    // ZK Proof validation happens here...

    // State is updated, but no prior check prevents reaching this point with a reused hash
    claimed[treasureHash] = true;
    claimsCount++;

    // Transfer logic...
}
```

Impact

Likelihood: High Exploitation requires no specialized tools. An attacker simply replays a single valid proof and hash. Since the contract fails to mark the specific hash as “used” effectively, the same transaction can be sent repeatedly until the contract is empty.

Impact: High Total Loss of Funds: An attacker can drain the entire 100 ETH reward pool using a single discovery.

Proof of Concept

This PoC demonstrates the Double-Spend exploit where a single proof drains the contract due to the faulty `_treasureHash` check.

Draining the Contract (Double-Spend):

```
function test_DrainContract_ClaimTenTimes() public {
    // 1. Load the proof, hash, and the original recipient
    (bytes memory proof, bytes32 treasureHash, address payable recipient) = _loadFixture();
```

```

// 2. Record initial balances
uint256 recipientInitialBalance = recipient.balance;
uint256 contractInitialBalance = address(hunt).balance;

// Ensure the contract starts with the full reward pool (100 ether)
assertEq(contractInitialBalance, 100 ether, "Contract should have 100 ether");

// 3. The Exploit: Loop 10 times using the exact same proof and hash
for (uint256 i = 0; i < 10; i++) {
    vm.prank(participant);
    hunt.claim(proof, treasureHash, recipient);
}

// 4. Verification: Check if the contract is completely empty
assertEq(address(hunt).balance, 0, "Contract should be fully drained");

// 5. Verification: Check if the recipient received the entire 100 ether from a single h
assertEq(recipient.balance, recipientInitialBalance + 100 ether, "Recipient should have
}

```

Recommended Mitigation

The fix requires a two-step approach to restore both security and functionality.

Solidity Fix: Change the check in `claim()` to reference the function parameter `treasureHash` instead of the uninitialized `_treasureHash`.

```

- bytes32 private immutable _treasureHash;
+ // Removed unused variable

- if (claimed[_treasureHash]) revert AlreadyClaimed(treasureHash);
+ if (claimed[treasureHash]) revert AlreadyClaimed(treasureHash);

```

[H-2] Duplicate Hash Constants in Noir Circuit Leads to Permanent Fund Lock

Description

Root Cause: The Noir circuit in `main.nr` contains duplicate entries in the `ALLOWED_TREASURE_HASHES` array (indices 8 and 9 share the same hash value). This creates a deadlock where only 9 unique treasures can be claimed, permanently locking funds.

```

// Baked-in set of 10 allowed treasure hashes (pedersen hashes).
global ALLOWED_TREASURE_HASHES: [Field; 10] = [
    1505662313093145631275418581390771847921541863527840230091007112166041775502,

```

```

-7876059170207639417138377068663245559360606207000570753582208706879316183353,
-5602859741022561807370900516277986970516538128871954257532197637239594541050,
2256689276847399345359792277406644462014723416398290212952821205940959307205,
10311210168613568792124008431580767227982446451742366771285792060556636004770,
-5697637861416433807484703347699404695743570043365849280798663758395067508,
-2009295789879562882359281321158573810642695913475210803991480097462832104806,
8931814952839857299896840311953754931787080333405300398787637512717059406908,
//@> -961435057317293580094826482786572873533235701183329831124091847635547871092,
//@> -961435057317293580094826482786572873533235701183329831124091847635547871092
];

```

Impact

Likelihood: High **Impact:** High Protocol Deadlock: The duplicate hash in Noir ensures the hunt can never be “completed” (9/10), which permanently bricks the `withdraw()` function for the owner, locking any remaining funds forever.

Proof of Concept

This PoC demonstrates the Protocol Deadlock where duplicate hashes in the Noir circuit prevent reaching the 10th claim.

Permanent Fund Lock (Duplicate Logic):

```

function test_LockedFundsDueToDuplicateHash() public {
    (bytes memory proof, bytes32 treasureHash, address payable recipient) = _loadFixture();

    vm.prank(participant);
    hunt.claim(proof, treasureHash, recipient);

    assertTrue(hunt.claimed(treasureHash));
    assertEquals(hunt.claimsCount(), 1);

    vm.prank(participant);
    vm.expectRevert(abi.encodeWithSelector(TreasureHunt.AlreadyClaimed.selector, treasureHash));
    hunt.claim(proof, treasureHash, recipient);

    vm.prank(owner);
    vm.expectRevert("HUNT_NOT_OVER");
    hunt.withdraw();

    assertGe(address(hunt).balance, 10 ether);
}

```

Recommended Mitigation

The fix requires a two-step approach to restore both security and functionality.

Noir Fix: Replace the duplicate hash at index 9 in `ALLOWED_TREASURE_HASHES` with a unique Pedersen hash. This ensures all 10 allowed hashes are distinct and allows the hunt to complete.

Medium

[M-1] Missing access control on `withdraw` function allows permissionless execution

Description

Normal Behavior: Administrative functions that handle the sweeping of remaining or unclaimed funds are strictly designed to be executed by the contract owner. This allows the protocol administrators to manage their treasury, calculate taxes, or move funds to a cold wallet on their own specific schedule.

Specific Issue: The `withdraw` function lacks the `onlyOwner` modifier. The function only checks if the hunt is over (`claimsCount >= MAX_TREASURES`). Once this condition is met, any external user, bot, or attacker can call the function. While the funds are hardcoded to be sent to the owner address (preventing theft), this permissionless execution removes the owner's control over the exact timing of the withdrawal, which can disrupt off-chain accounting systems or trigger unwanted tax events for the protocol.

```
// ----- admin functions -----

/// @notice Allow the owner to withdraw unclaimed funds after the hunt is over.
// @> Root cause: Missing onlyOwner modifier allows anyone to trigger the sweep.
function withdraw() external {
    require(claimsCount >= MAX_TREASURES, "HUNT_NOT_OVER");

    uint256 balance = address(this).balance;
```

Impact

Likelihood: Any user or MEV bot can execute this function immediately after the final treasure is claimed.

Impact: No direct loss of funds, as the ETH is securely routed to the owner address. Protocol griefing through forced operational timing. The owner loses control over when the transaction occurs.

Proof of Concept

The following Foundry test demonstrates how an unauthorized user can force the withdrawal:

1. The test simulates a state where all 10 treasures have been successfully claimed.
2. A random user (`attackerAddress`) calls the `withdraw()` function.
3. Because there is no `msg.sender == owner` check, the transaction succeeds.
4. The remaining contract balance is forcefully pushed to the owner without their consent.

```
function testAnyUserCanForceWithdrawal() public {
    // 1. Simulate the end of the game
    vm.store(address(treasureHunt), bytes32(uint256(1)), bytes32(uint256(10))); // Set claim

    // 2. Prank as an unauthorized user
    address randomUser = address(0x1337);
    vm.prank(randomUser);

    // 3. Execute the admin withdrawal function
    treasureHunt.withdraw(); // @> Succeeds without reverting

    // 4. Verify funds were swept
    assertEq(address(treasureHunt).balance, 0);
}
```

Recommended Mitigation

Explanation of the fix: Implementing the existing `onlyOwner` modifier ensures that the EVM checks `msg.sender == owner` before executing the logic. This restores full control to the protocol administrators regarding when the remaining treasury is swept.

```
/// @notice Allow the owner to withdraw unclaimed funds after the hunt is over.
- function withdraw() external {
+ function withdraw() external onlyOwner {
    require(claimsCount >= MAX_TREASURES, "HUNT_NOT_OVER");

    uint256 balance = address(this).balance;
```

Low

[L-1] Missing Zero-Address Validation in `updateVerifier`

Description

Normal Behavior: When updating critical external dependencies, such as the address of a ZK-proof Verifier contract, the input parameters must be validated to ensure they are not set to the zero address

(0x00). This prevents the main contract from attempting to execute calls on non-existent bytecode.

Specific Issue: The `updateVerifier` function accepts an `IVerifier` type address and assigns it directly to the `verifier` state variable without validating the input. If the protocol owner accidentally passes `address(0)` during an emergency update, the state is saved. Subsequently, when any user attempts to call the `claim` function, the contract will execute `verifier.verify(proof, publicInputs)`. Because high-level Solidity calls to EOAs or the zero address revert if the target has no code, the entire `claim` function will consistently revert. This causes a complete Denial of Service (DoS) for the core application logic until the owner submits another transaction to fix it.

```
/// @notice In case of a bug, allow the owner to update the verifier address.
function updateVerifier(IVerifier newVerifier) external {
    require(paused, "THE_CONTRACT_MUST_BE_PAUSED");
    require(msg.sender == owner, "ONLY_OWNER_CAN_UPDATE_VERIFIER");
    // @> Root cause: newVerifier is assigned without checking against address(0)
    verifier = newVerifier;

    emit VerifierUpdated(address(newVerifier));
}
```

Impact

Likelihood: This relies entirely on an administrative mistake (human error) during a contract upgrade or emergency intervention.

Impact: A complete Denial of Service (DoS) for the `claim` function. Users will be entirely unable to claim their rewards. Funds and rewards will remain temporarily locked in the contract until the administrator recognizes the mistake, unpauses, pauses again, and executes another update transaction with a valid contract address.

Proof of Concept

The following steps outline the exact execution flow that leads to the DoS:

1. The owner pauses the contract to perform an upgrade.
2. The owner accidentally submits `address(0)` as the parameter for `updateVerifier`.
3. The transaction succeeds, setting the `verifier` state variable to `0x00...00`.
4. The owner unpauses the contract.
5. A user submits a valid ZK proof to the `claim` function.
6. The EVM attempts to call `verify()` on `address(0)`. Because it is a high-level call to an address with no deployed contract code, the EVM immediately reverts the transaction.

```

function testUpdateVerifierToZeroAddressCausesDoS() public {
    // 1. Owner pauses the contract
    vm.prank(owner);
    treasureHunt.pause();

    // 2 & 3. Owner accidentally passes the zero address
    vm.prank(owner);
    treasureHunt.updateVerifier(IVerifier(address(0)));

    // 4. Owner unpauses
    vm.prank(owner);
    treasureHunt.unpause();

    // 5 & 6. Valid user attempts to claim, but the call to address(0) reverts
    vm.prank(user);
    vm.expectRevert(); // High-level call to non-contract address reverts
    treasureHunt.claim(validProof, validTreasureHash, payable(user));
}

```

Recommended Mitigation

Explanation of the fix: Adding a specific `if` statement or `require` check against `address(0)` acts as an automated safeguard against human error. This mirrors the exact safety check that the developer originally included in the constructor, ensuring consistency across the codebase.

```

function updateVerifier(IVerifier newVerifier) external {
    require(paused, "THE_CONTRACT_MUST_BE_PAUSED");
    require(msg.sender == owner, "ONLY_OWNER_CAN_UPDATE_VERIFIER");
+   if (address(newVerifier) == address(0)) revert InvalidVerifier();
    verifier = newVerifier;

    emit VerifierUpdated(address(newVerifier));
}

```